

OPERATING AGENT SYSTEM

Universal Agent Operating System & Control Plane

Comprehensive Technical Blueprint, Architectural Reference, Capabilities Catalog, and Enterprise Strategy for Autonomous Multi-Agent Software Engineering.

Version: 2.2.1

Author: Nihir Patel

License: MIT Forever

Classification: Technical Whitepaper

69 Specialized Agents

Autonomous subagents for architecture, TDD, code review, build repair, security, and ML pipelines.

286 Production Skills

Modular, battle-tested engineering workflows with YAML metadata across all languages and frameworks.

12 Supported Harnesses

Cross-platform runtime parity: Claude Code, Codex, Cursor, Gemini, OpenCode, Zed, Kimi, etc.

40,000+ Stars & Trending #1

Ecosystem standard for AI agent harnesses, memory vault, and enterprise control-plane orchestration.

KEY PLATFORM CAPABILITY: THE OAS PLATFORM AXIOM

"Optimize the context window. Persist everything else." OAS elevates AI coding from brittle, conversational "vibe coding" into a deterministic, test-driven, self-healing software engineering discipline. It orchestrates multi-agent swarms with mathematical guarantees against context collapse, supply-chain poisoning, and concurrent code regressions.

DOCUMENT CONTENTS & EXECUTIVE OUTLINE

- | | | |
|---------------------------------------|--|---------------------------------------|
| 1. Executive Summary & Philosophy | 5. 69 Autonomous Agents Taxonomy | 9. AgentShield Supply Chain Security |
| 2. Mission, Vision & Industry Need | 6. 286 Workflow Skills Ecosystem | 10. OAS Studio & Cloud Control Plane |
| 3. The 6 Critical Problems OAS Solves | 7. OAS Memory Vault & Unified Context | 11. User Benefits & Enterprise Impact |
| 4. 3-Layer Operating Architecture | 8. TCAS Airspace Collision Deconfliction | 12. 12-Harness Support & Roadmap |

1. Executive Summary & System Overview

The Operating Agent System (OAS) is a production-grade, harness-native agent operating system designed to bring engineering rigor, determinism, and enterprise security to AI-assisted software development. Unlike basic autocomplete plugins, superficial prompt wrappers, or monolithic proprietary IDEs, OAS provides a universal meta-harness layer that unifies specialized agent orchestration, workflow skills, deterministic lifecycle hooks, durable cross-session memory, and real-time execution control.

Currently deployed at version 2.2.1, OAS has emerged as one of the most widely adopted open-source agent frameworks in the world, holding the #1 global trending position on GitHub with over 40,000 stars and multi-thousand weekly downloads across npm. It acts as a force multiplier for software teams by converting raw Foundation Models (Claude 3.7 Sonnet, GPT-4o, Gemini 2.5, DeepSeek-R1, and open-source models) into disciplined software engineering teams.

The Core OAS Philosophy

OAS was founded on five non-negotiable engineering principles designed to prevent the common failure modes of autonomous coding agents:

- ? **1. Agent-First Specialization:** Never force a single prompt to do everything. Complex tasks are decomposed and routed to domain-specific agents (planners, architects, TDD guides, language reviewers, build resolvers, security auditors).
- ? **2. Test-Driven Development (TDD):** Mandatory 80%+ test coverage. Every feature and bugfix must strictly follow the Red-Green-Refactor loop. Code without tests is rejected at the gate.
- ? **3. Security-First & Zero Trust:** All user input is treated as untrusted. Hardcoded secrets, unvalidated SQL/HTML, prompt injection vectors, and untrusted dependencies are actively intercepted and blocked by AgentShield.
- ? **4. Immutability & Pure State:** Code and state modifications must create new objects and return copies with changes applied rather than mutating shared memory or global state, preventing catastrophic concurrency bugs.
- ? **5. Plan Before Execute:** No destructive edits without an approved plan. The agent decomposes goals into verified milestones, reviews them with human operators via Plan Canvas, and executes sequentially.

NOTE: THE DICHOTOMY

Raw AI models are probabilistic autocomplete engines. Software engineering is a deterministic discipline where a single misplaced token breaks production. OAS bridges this chasm by constraining probabilistic intelligence with deterministic operating system primitives.

2. Mission and Vision

The software industry is experiencing an unprecedented transition: developers are moving from manual coding to supervising autonomous AI agents. However, this transition has exposed deep structural flaws in existing tooling. Current AI tools operate in disconnected silos, suffer from severe amnesia between chat sessions, degrade rapidly as context grows, and introduce massive supply-chain security risks.

The OAS Mission

To build the universal, open-standard operating system for AI software engineering?giving every developer, engineering team, and enterprise an immutable, test-driven, and secure runtime that works seamlessly across any AI coding harness, model, or IDE.

The OAS Vision

OAS envisions a future where software development is transformed from manual token-by-token typing into multi-agent collaborative engineering. In this future:

- ? **Autonomous Engineering Swarms:** Specialized agents work asynchronously across isolated git worktrees, implementing features, writing regression tests, reviewing diffs, and fixing build errors in parallel.
- ? **Durable Organizational Memory:** Every architectural decision, bug diagnosis, and domain learning is automatically captured into a shared, inspectable memory vault that follows developers across sessions and platforms.
- ? **Harness Freedom & Anti-Lock-in:** Engineers are never locked into a single proprietary subscription. The same skills, rules, memory, and agents execute with identical behavior whether running in Claude Code, Codex, Cursor, Gemini, OpenCode, or Zed.
- ? **Provable Software Correctness:** Code generation is governed by automated mathematical verification, formal evaluation harnesses, and continuous supply-chain auditing, ensuring that software built with AI meets or exceeds human safety standards.

3. What Problems Is OAS Solving?

Modern AI development tools suffer from six systemic problems that prevent them from scaling beyond toy demos into mission-critical production systems. OAS was architected from the ground up to solve each of these failure modes:

Problem 1: The 'Vibe Coding' Trap & Brownfield Collapse

The vast majority of AI coding tools encourage 'vibe coding'—generating hundreds of lines of code based on vibes and conversational prompts without tests or architectural planning. In greenfield projects this feels fast, but in brownfield production codebases it invariably causes silent regressions, broken contracts, and cascading technical debt. OAS enforces the Red-Green-Refactor TDD cycle and architectural planning before a single line of production code is touched.

Problem 2: The Context Cliff & Cognitive Degradation

Large Language Models exhibit severe cognitive degradation as conversation transcripts swell past 80,000–120,000 tokens (known as the 'Context Cliff'). Models begin hallucinating nonexistent functions, forgetting earlier instructions, and repeating mistakes. OAS features the Strategic Context Compactor, which monitors token budgets in real time and compacts transient logs while persisting critical architectural facts to disk.

Problem 3: Harness Fragmentation & Vendor Lock-in

Engineering teams are fragmented across tools: frontend engineers use Cursor, backend teams use Codex or OpenCode, terminal power-users use Claude Code, and cloud engineers use Gemini. Previously, each tool required separate prompts, custom rules, and proprietary configuration. OAS acts as a Universal Meta-Harness: write a skill or rule once, and it automatically works across all 12 supported harnesses.

Problem 4: Cross-Session Amnesia & Fragmented State

Every time an AI chat session ends, its accumulated knowledge is wiped out. If an agent spent two hours understanding a subtle concurrency bug in your authentication service, the next agent tomorrow starts from zero. OAS features the OAS Memory Vault (backed by SQLite and standardized Markdown frontmatter), enabling agents to save lessons, architectural decisions, and handoff state across sessions and different harnesses.

Problem 5: Supply Chain Attacks & Weaponized Agent Tools

In early 2026, research disclosures (CVE-2025-59536, CVE-2026-21852, and Snyk's ToxicSkills study) revealed that 36% of public agent skills contained prompt injection or malicious execution payloads. Malicious git repositories could execute code or exfiltrate API keys before users even confirmed trust dialogs. OAS integrates AgentShield, which actively scans hooks, prompts, MCP configs, and dependencies for supply-chain indicators of compromise (IOCs).

Problem 6: Runaway Autonomous Loop Stalls & Token Burn

When agents are run in autonomous loops (e.g., Ralphinho RFC pipelines or overnight batch tasks), they frequently get trapped in infinite retry loops, repeating the same broken command and burning hundreds of dollars in API credits. OAS includes the Loop Operator agent, TCAS Airspace deconfliction, and stale loop monitors that detect repetitive error states and trigger human-in-the-loop intervention.

4. How the Platform Works: The 3-Tier Operating Architecture

OAS operates as a full-stack agent operating system structured into three distinct, highly coordinated layers. This modular design allows organizations to run OAS as a complete end-to-end platform or selectively adopt the layers that fit their existing toolchain:

The 3-Layer System Model

- Layer 1: Meta-Harness & Skill Ecosystem:** The foundational layer containing 69 specialized agents, 286 workflow skills, 94 command entrypoints, deterministic lifecycle hooks, and language-specific rules. It provides universal manifest resolution, allowing any supported harness (Claude Code, Codex, Cursor, Gemini, etc.) to discover and execute standardized OAS assets.
- Layer 2: Dedicated OAS Agent Runtime Engine:** The local orchestration and execution engine implemented in Node.js and TypeScript (``packages/engine``, ``packages/db``, ``packages/parser``). It includes the `AgentDagScheduler` for multi-agent DAG pipelines, the `ExecutionSandbox` for isolated terminal execution, the `StrategicCompactor` for token budgeting, the `UniversalModelGateway` for multi-model routing, and the `WorktreeRunner` for git isolation.
- Layer 3: Enterprise Control Plane & Agentic IDE:** The visual operator surface and real-time streaming control plane (``apps/web`` and ``apps/api``). It features a browser-based Studio UI with live DAG visualization, real-time Server-Sent Events (SSE) telemetry, interactive Knowledge Graph navigation, Plan Canvas Pro for collaborative plan review, and multi-session management.

Detailed Engine Subsystems (``packages/engine``)

Subsystem	Component	Responsibility & Mechanism
DAG Scheduler	<code>scheduler.js</code>	Constructs dependency execution graphs for multi-agent workflows with event-driven pause, resume, and abort controls.
Execution Sandbox	<code>sandbox.js</code>	Enforces path-confined file access and command execution, preventing runaway scripts from escaping the project root.
Strategic Compactor	<code>compactor.js</code>	Monitors token consumption against model budget thresholds (e.g. 200k tokens), triggering structured summaries.
Model Gateway	<code>llm-gateway.js</code>	Universal abstraction layer routing requests across Anthropic, OpenAI, Gemini, and open-source models with failover.
Worktree Runner	<code>worktree-runner.js</code>	Provisions isolated Git worktrees for concurrent agent branches, eliminating workspace file collisions.
Command Runner	<code>command-runner.js</code>	Executes 94 slash commands and pipelines, bridging human intent to agent task execution.
Parser & Sync	<code>parser.js</code>	Indexes all agents, skills, hooks, and MCP servers into an in-memory knowledge graph and SQLite database.
Memory Store	<code>packages/db</code>	Local SQLite database (<code>`.oas-store.json`</code>) tracking sessions, telemetry, agent step events, and operator interventions.

KEY PLATFORM CAPABILITY: ARCHITECTURAL PRINCIPLE: ZERO CLOUD LOCK-IN

The entire OAS runtime engine executes 100% locally on the developer's machine or self-hosted server. Telemetry and state are stored in a local SQLite file. No private code, customer prompts, or API keys are ever transmitted to third-party servers without explicit user configuration.

5. The Complete Capabilities Catalog: 69 Specialized Agents

A foundational insight of OAS is that general-purpose prompts fail at specialized tasks. A prompt optimized for high-level software architecture makes poor decisions during low-level memory debugging, and a prompt tuned for aggressive code generation frequently overlooks subtle security flaws. OAS solves this by providing 69 specialized agents, each configured with domain-specific system instructions, tool restrictions, and validation rubrics.

Agent Taxonomy & Core Directory

Agent ID	Domain & Specialization	Primary Purpose & Activation Criteria
planner	Planning & Strategy	Implementation planning, breaking complex features into phases, risk analysis.
architect	System Architecture	System design, scalability, service decomposition, and architectural decisions.
tdd-guide	Test-Driven Development	Enforcing Red-Green-Refactor cycles, writing unit/integration tests before code.
code-reviewer	General Code Review	Post-implementation quality review, maintainability, and standard compliance.
security-reviewer	Vulnerability & Security	Auditing authentication, secrets, OWASP top 10, and injection vulnerabilities.
spec-miner	Brownfield Onboarding	Extracting specifications and data contracts from legacy brownfield codebases.
build-error-resolver	Universal Build Fixer	Diagnosing and fixing compilation, bundling, and TypeScript type errors.
e2e-runner	End-to-End Testing	Generating, maintaining, and running Playwright browser automation tests.
refactor-cleaner	Code Maintenance	Eliminating dead code, unused exports, duplicate abstractions, and tech debt.
doc-updater	Documentation & Maps	Synchronizing READMEs, API codemaps, and Architecture Decision Records (ADRs).
typescript-reviewer	TypeScript / JavaScript	Specialist review for TS 5.x, React, Node, Next.js, and strict type safety.
python-reviewer	Python & Ecosystem	Code quality, PEP 8, async patterns, and library idioms for Python services.
go-reviewer	Go Systems	Go concurrency patterns (goroutines, channels), interfaces, and error handling.
go-build-resolver	Go Build Repair	Resolving Go compilation, module dependency, and linking failures.

Agent Taxonomy (Continued: Systems, Languages & Infrastructure)

Agent ID	Domain & Specialization	Primary Purpose & Activation Criteria
rust-reviewer	Rust & Memory Safety	Reviewing ownership, borrowing, lifetime annotations, and unsafe code boundaries.
rust-build-resolver	Rust Compiler Errors	Interpreting rustc borrow-checker errors and resolving cargo build breaks.
cpp-reviewer	C / C++ Systems	Modern C++20/23, RAII, pointer safety, undefined behavior, and memory leaks.
cpp-build-resolver	C/C++ Build Systems	CMake, Make, Clang/GCC compiler error resolution, and linking errors.
java-reviewer	Java & Spring Boot	Enterprise Java patterns, Spring Boot configuration, DI, and Hibernate/JPA.
java-build-resolver	Java Build Tooling	Maven and Gradle build resolution, dependency conflicts, and JVM tuning.
kotlin-reviewer	Kotlin & Mobile	Kotlin Coroutines, Flows, Android Jetpack Compose, and KMP architecture.
django-reviewer	Django & DRF	Django ORM efficiency (N+1 query elimination), migrations, and REST APIs.
django-build-resolver	Django Operations	Resolving Django migration clashes, collectstatic failures, and startup errors.
database-reviewer	PostgreSQL & Supabase	Database schema design, indexing strategies, query plan optimization (EXPLAIN).
mle-reviewer	Machine Learning Eng.	ML pipeline validation, data contracts, model eval, serving, and drift detection.
rag-pipeline-reviewer	RAG & Vector Search	Chunking strategies, embedding retrieval quality, reranking, and RAGAS evals.
loop-operator	Autonomous Execution	Monitoring autonomous agent loops, detecting stalls, managing token budgets.
harness-optimizer	Harness Performance	Tuning harness configs for reliability, maximum throughput, and low token cost.

Autonomous Agent Orchestration

OAS agents are not designed to work in isolation. The system employs proactive multi-agent orchestration, where agents automatically delegate to one another based on lifecycle stage:

- ? **Feature Initiation:** When a user requests a complex feature, OAS automatically invokes the planner to establish requirements and an implementation plan before code is touched.
- ? **Development Cycle:** The planner hands off to tdd-guide, which writes failing tests (RED) and prompts for minimal code (GREEN).
- ? **Review & Quality Gate:** Once implementation completes, code-reviewer and language-specific reviewers (e.g., typescript-reviewer) evaluate maintainability while security-reviewer scans for vulnerabilities.
- ? **Failure Recovery:** If a build or type error occurs at any point, control automatically transfers to build-error-resolver, which isolates and fixes the error incrementally before returning control.

6. The 286 Workflow Skills Ecosystem

While agents represent active execution personas, Skills represent procedural domain knowledge and workflows. Every OAS skill lives in `skills/<name>/SKILL.md` and contains structured YAML frontmatter (name, description, origin, metadata), actionable step-by-step guidance, tested code patterns, and explicit 'When to Use' criteria.

With 286 production skills, OAS provides the most comprehensive AI development catalog in existence, spanning modern web development, systems programming, distributed architectures, machine learning, bioinformatics, and enterprise operations.

Skill Categorization & High-Impact Domains

- ? **Frontend & UI Engineering:** React, Next.js 16+ Turbopack, Vue, Nuxt 4, Svelte, Tailwind, Liquid Glass, Motion Foundations, Accessibility (a11y), CWV optimization (LCP, INP).
- ? **Backend & Distributed Systems:** Express, NestJS, FastAPI, Django, Spring Boot, Quarkus, Golang patterns, Microservices, Hexagonal architecture, ClickHouse, Postgres, Redis, Prisma.
- ? **Test-Driven Dev & Verification:** TDD workflow, Playwright E2E testing, AI regression testing, verification loop, checkpointing, contract-first APIs, test coverage analysis.
- ? **Cloud, DevOps & Infrastructure:** Docker patterns, Kubernetes, Flox environments, Uncloud self-hosting, homelab networking (Pi-hole, WireGuard, VLANs), serverless architectures.
- ? **AI, Machine Learning & RAG:** MLE production pipelines, RAG pipeline evaluation, token budget advisory, Prompt optimizer, on-device foundation models, RecSys architecture.
- ? **Biomedical & Life Sciences:** AlphaFold DB structural analysis, Ensembl, UniProt, ChEMBL, PubMed, ClinicalTrials.gov, dbSNP, gnomAD, Foldseek, PDB biomolecular modeling.
- ? **Security & Supply Chain:** AgentShield scan, supply chain incident response, DeFi AMM security, HIPAA/PHI compliance, memory leak debugging, secret detection.
- ? **Multi-Agent & Meta-Operations:** DMUX multi-agent orchestration, continuous agent loops, autonomous loops, unified memory vault, plan canvas, worktree orchestration.

Skill Anatomy & Standards

Every skill adheres to strict quality and safety guidelines defined in `RULES.md`:

```
---
name: unified-memory
description: Share durable context between Claude, Codex, Cursor, and other harnesses.
metadata:
  origin: OAS
---
# Unified Memory Workflow
## Runtime Prerequisite: Requires oas-universal CLI
## When To Use: Save durable context, hand off work across harnesses, resume prior tasks.
## Scopes: project (.oas/memory/project/), team (.oas/memory/team/), user (~/.oas/memory/)
```


7. Flagship Innovation: OAS Memory Vault

A central architectural innovation of OAS is the Memory Vault (``oas memory`` CLI and ``oas-memory-mcp``). Historically, AI agents have been ephemeral: every new session begins with a blank slate, and context cannot be shared between different tools (e.g., passing work from Claude Code to Codex). The OAS Memory Vault provides a universal, inspectable, and secure persistence layer.

The 3-Tier Storage Scopes

Scope	Storage Location	Governance & Visibility Rules
Project Scope	<code><repo>/oas/memory/project/</code>	Repo-local ephemeral context. Protected by a fail-closed <code>`.gitignore`</code> to prevent accidental secret commits.
Team Scope	<code><repo>/oas/memory/team/</code>	Durable architectural context intended for version control, team review, and Git PR sharing.
User Scope	<code>~/oas/memory/</code>	Operator-level context that follows the individual developer across all repositories on their machine.

Memory Schema & Cross-Harness Handoffs (``oas.memory.v1``)

Memories are stored as human-readable Markdown files with strict YAML frontmatter conforming to ``oas.memory.v1``. This allows any developer to inspect, edit, or delete memories using standard tools like ``cat``, ``grep``, or VS Code:

```
---
schema: oas.memory.v1
id: mem-2026-09-07-auth-migration
title: Authentication Migration to PKCE
kind: handoff
source_harness: claude-code
target_harness: codex
trust: unreviewed
tags: [auth, security, migration]
---

## Handoff Summary
Unit tests in tests/auth.test.js pass. Pending task: update DRF views to validate PKCE challenge.
```

CRITICAL SECURITY DIRECTIVE: FAIL-CLOSED SECURITY IN THE VAULT

The Memory Vault enforces a strict fail-closed security boundary: memories recalled into agent context are treated as untrusted data rather than executable instructions. This prevents memory poisoning attacks (where an adversary implants malicious commands into persistent memory to hijack future agent sessions).

8. TCAS Airspace Collision Deconfliction (Layer 4)

When running multi-agent swarms or parallel autonomous loops (e.g., using ``dmux`` or background workers), a critical danger arises: multiple agents editing the same files simultaneously, causing merge conflicts, overwritten code, and race conditions. OAS solves this with TCAS (Traffic Collision Avoidance System)?a Layer 4 Airspace Deconfliction engine.

How TCAS Protects the Workspace

- ? **1. File Proximity Probing:** Before an agent begins editing a file or module, TCAS checks active agent leases. If another agent is currently modifying the same file tree, TCAS issues an immediate proximity alert.
- ? **2. Worktree Isolation:** When parallel tasks are detected, TCAS automatically spins up temporary, isolated Git worktrees (``scripts/orchestrate-worktrees.js``). Each agent works on a private filesystem branch, completely isolated from concurrent edits.
- ? **3. Semantic Lock Arbitration:** If two agents require write access to the same shared interface or schema, TCAS acts as an arbiter, pausing the secondary agent until the primary agent commits and passes unit tests.
- ? **4. Collision Deconfliction:** Upon task completion, TCAS merges the isolated worktree back into the main branch, verifying that tests still pass and no regressions were introduced.

Strategic Context Compaction

In addition to spatial deconfliction, OAS manages temporal resource limits through the Strategic Context Compactor (``packages/engine/src/compactor.js``). When an agent session approaches the 200,000 token limit, the compactor:

- ? **Strips Ephemeral Traces:** Removes raw terminal outputs, large JSON tool responses, and redundant diffs that have already been validated.
- ? **Distills Decisions:** Extracts key architectural decisions, file changes, and pending goals into structured summaries.
- ? **Preserves Momentum:** Re-injects the compacted state into a clean context window, allowing tasks to run indefinitely without hitting the context cliff.

9. AgentShield: Enterprise Supply Chain Security

In 2025 and 2026, the rise of autonomous coding tools created a new attack vector: AI supply-chain poisoning. Attacks such as CVE-2025-59536 (Remote Code Execution via poisoned project configs) and CVE-2026-21852 (API key exfiltration via redirected base URLs) demonstrated that simply opening an untrusted repository in an AI tool could compromise an engineer's machine. OAS addresses this with AgentShield—a dedicated security suite built directly into the operating system.

The 5 Layers of AgentShield Protection

Protection Layer	Scan Surface	Threat Mitigated
1. IOC Scanner	<code>`scripts/ci/scan-supply-chain-iocs.js`</code>	Scans project files for known malicious domains, URLs, base URL overrides, and leaked credentials.
2. Prompt Guard	<code>`packages/engine/src/sandbox.js`</code>	Detects indirect prompt injection payloads hidden inside markdown docs, PR comments, and email attachments.
3. Hook Auditor	<code>`scripts/ci/validate-hooks.js`</code>	Validates that lifecycle hooks do not execute arbitrary shell commands without explicit user permission.
4. MCP Gatekeeper	<code>`mcp.json`</code> & server configs	Prevents shadow MCP servers from auto-approving dangerous tools or silently exfiltrating source code.
5. Secret Interceptor	PreToolUse Hook Engine	Strips API keys, bearer tokens, SSH keys, and system paths from agent tool outputs before they reach the model.

The Simon Willison 'Lethal Trifecta'

AgentShield is architected specifically to dismantle the 'Lethal Trifecta' of AI vulnerabilities:

- ? **Private Data Access:** The agent has read access to proprietary source code and local secrets.
- ? **Untrusted Content Processing:** The agent reads foreign input (untrusted GitHub PRs, external docs, web search results).
- ? **External Communication:** The agent has access to tools that can make network requests or push git commits.

When all three conditions exist, prompt injection transforms from a curiosity into remote code execution and data exfiltration. AgentShield breaks this chain by strictly sandboxing network communication, sanitizing external inputs, and sequestering secrets in memory.

10. OAS Studio & Enterprise Cloud Control Plane

While terminal-based CLI tools are ideal for headless execution, complex multi-agent workflows require visual observability. OAS includes OAS Studio (`scripts/oas-studio.js` and `apps/web`), an enterprise-grade web control plane and visual agentic IDE running locally at `http://127.0.0.1:3458/`.

Studio Architecture & Capabilities

View / Subsystem	Technology	Operational Function
Execution DAG	SVG / Canvas + SSE	Visualizes real-time dependency graphs of executing agents, showing task status, token burn, and pause/resume controls.
Live Workspace	Real-time Terminal	Interactive console allowing operators to monitor live stdout/stderr, inspect tool calls, and provide human feedback.
Knowledge Graph	Force-Directed Graph	Interactive 3D/2D visualization linking 69 agents, 286 skills, 94 commands, and MCP servers with searchable node relationships.
Plan Canvas Pro	Interactive Canvas	Browser-based canvas for reviewing implementation plans, allowing operators to annotate items, chat, and click 'Approve'.
Capabilities Catalog	Virtual List UI	Comprehensive, filterable library of all installed skills, agents, commands, and MCP configs with instant search.
Memory & Telemetry	SQLite Inspector	Visual explorer for the OAS Memory Vault and SQLite state store, displaying session history and telemetry metrics.
Studio Builder	Visual Workflow Editor	Drag-and-drop agent pipeline builder for chaining agents, defining fallback triggers, and creating custom DAGs.

Real-Time Streaming Engine (`apps/api/src/server.js`)

The control plane communicates with the execution engine via a high-performance Server-Sent Events (SSE) streaming engine. It emits fine-grained events for every step in an agent's lifecycle:

- ? **agent:step:** Emitted whenever an agent performs reasoning, invokes a tool, or produces code modifications.
- ? **agent:intervention:paused:** Triggered when an agent detects an ambiguous requirement or dangerous command, pausing execution for operator review.
- ? **agent:human_in_the_loop:feedback:** Allows the human operator to inject course-correcting instructions directly into the running agent without restarting the session.

11. Value Proposition: How OAS Empowers Users

OAS delivers distinct, measurable value across every tier of the software engineering hierarchy:

For Individual Developers & Freelancers

Eliminates the tedious cognitive overhead of writing boilerplate, unit tests, and documentation. Developers act as technical directors, approving plans on Plan Canvas while specialized agents write tests, implement code, and fix build errors automatically. Delivers 10x engineering velocity without sacrificing code quality.

For Tech Leads & Software Architects

Enforces team-wide architectural discipline automatically. Standardized rules (immutability, 80%+ test coverage, clean architecture) are strictly verified by automated code-reviewers and security-reviewers before code can be merged, eliminating architectural drift across large teams.

For Enterprise Engineering & CISO Teams

Solves the compliance, security, and governance nightmare of 'shadow AI.' AgentShield eliminates supply-chain risks, verifies dependencies, and sanitizes secrets. Complete audit trails in SQLite provide provable governance for regulated industries (healthcare HIPAA, financial compliance).

For Autonomous Loop & AI Engineers

Provides the bulletproof infrastructure needed to run safe, multi-hour autonomous loops. TCAS prevents file collisions, the Strategic Compactor prevents token exhaustion, and loop monitors eliminate runaway retries, allowing safe overnight feature development.

The Open-Source Business Model & Platform Value Loop

OAS follows the proven open-source infrastructure playbook (similar to Linux, PostgreSQL, or Kubernetes):

- 🔍 **Free Forever Open-Source Core (MIT):** The full meta-harness layer, all 69 agents, 286 skills, memory vault, and local CLI remain MIT-licensed open source forever. Anyone can clone, inspect, and use it freely.
- 🔍 **OAS Pro & Hosted Control Plane (\$19/seat/mo):** For private enterprise repositories, OAS Tools provides a hosted GitHub App, managed multi-agent queues, cloud team memory synchronization, and compliance audit reporting.
- 🔍 **Compute & Ecosystem Partnerships:** Strategic partnerships with high-performance compute and evaluation providers (Itô Markets for decentralized compute, Fal.ai for media, CodeRabbit and Greptile for automated review) fund continuous full-time maintenance.

12. Compatibility Matrix & Getting Started Guide

OAS provides native compatibility and adapters across 12 modern AI harnesses and editors:

Harness / Editor	Integration Level	Supported Capabilities
Claude Code	Native Full Support	Full 69 agents, 286 skills, 94 commands, lifecycle hooks, memory vault, and guided plugin setup.
Codex (OpenAI)	Native Sync Path	Full agent sync, memory vault, worktree execution, and MCP configuration.
Cursor IDE	Adapter Support	Rules, cursorrules generation, memory vault CLI, and skill workflow lookup.
Google Gemini	Adapter Support	Adapted agents, system instructions, MCP tools, and memory integration.
OpenCode	Plugin Support	Full plugin pack, commands, skills, and model gateway integration.
Zed Editor	Adapter Support	Slash commands, context server configuration, and rules enforcement.
Kimi Code	Native Guided Setup	Full guided install, skills pack, and hook profile integration.
Hermes	Adapter Support	Memory vault bridge, cross-harness handoffs, and skill discovery.
Pi	Extension Support	Pi extension bridge (`.pi/extensions`), prompts, and skills.
Antigravity IDE	Native Plugin Support	Eager/lazy MCP servers, custom skills, rules, and background tasks.
Qwen Code	Adapter Support	Rules pack, prompt templates, and selective skill manifests.
GitHub Copilot	Rules Adapter	Repo-level instructions (`.github/copilot-instructions.md`) and coding standards.

Quick-Start Installation & Verification

Getting started with OAS is accomplished via the universal Node.js CLI:

```
# 1. Guided Multi-Harness Setup (Configures Claude Code, Codex, or Kimi):
npx oas-universal setup

# 2. Launch the Enterprise Studio & Cloud Control Plane UI:
npx oas-universal control-pane # or: node scripts/oas-studio.js

# 3. Initialize the OAS Memory Vault in your repository:
oas memory init --scope project

# 4. Run System Diagnostic & Health Check:
oas doctor
```

Conclusion

Operating Agent System (OAS) v2.2.1 represents the maturation of AI-driven software development from erratic, unstructured autocomplete prompts into an enterprise-grade, deterministic engineering discipline. By uniting specialized autonomous agents, comprehensive workflow skills, durable cross-session memory, uncompromising supply-chain security, and a real-time visual control plane, OAS provides the foundational operating system that will power the next decade of software engineering.